AI-Based Programming

Lab 4: Evaluation, Experiment Tracking & Testing

Eng. Alshaymaa Abdelhady

Table of contents

01 Introduction to Evaluation

02 Evaluation Metrics

03 Evaluation Module

04 Basic Unit Testing

05 Track Experiments & Configurations

06 Reproducible Evaluation Pipeline

01 Introduction to Evaluation

Why Model Evaluation is Important ?

- **Model evaluation is a critical step in the machine learning workflow.**
  It assesses how well a trained model performs and whether it can generalize to new, unseen data.

- Without proper evaluation, a model may achieve high performance on training data while failing to perform reliably in real-world scenarios

Why Model Evaluation is Important ? (Cont.)

1. Ensuring Generalization
Evaluation helps determine whether the model has learned meaningful patterns rather than memorizing the training dataset (overfitting).

2. Validating Performance on Unseen Data
By testing the model on validation or test datasets, we can estimate how it will perform on new inputs.

3. Enabling Fair Model Comparison
Evaluation metrics such as accuracy and F1-score allow objective comparison between different models and algorithms.

Why Model Evaluation is Important ? (Cont.)

4. Supporting Model Optimization
Performance analysis helps identify weaknesses and guides improvements through hyperparameter tuning or feature selection.

5. Increasing Reliability and Trustworthiness
A well-evaluated model provides confidence that it can be safely used in real-world applications.

Therefore, systematic evaluation is essential for selecting robust machine learning models and ensuring reliable deployment in practical applications.

02 Evaluation Metrics

Evaluation Metrics

- Evaluation metrics are quantitative measures used to assess the performance of machine learning models. They help determine how well a model generalizes to unseen data and whether it produces reliable predictions.

- Different metrics are used depending on the **type of task**, such as classification or regression.

Classification Metrics (Categorical Outputs)

Accuracy

- Measures the proportion of correct predictions out of all predictions.
- Works best when the dataset is **balanced between classes**

Precision

- Measures the proportion of correctly predicted positive instances among all predicted positives.
- Important when **false positives are costly**

Recall (Sensitivity)

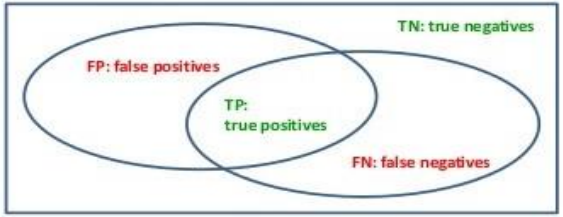
- Measures the proportion of actual positive cases that were correctly identified.
- Important when **missing positive cases is risky**.

F1-Score

- Harmonic mean of precision and recall.
- Useful when dealing with **imbalanced datasets**

Classification Metrics (Categorical Outputs) (Cont.)

Accuracy, Precision, Recall, and F-measure



Precision:

$$p = \frac{TP}{TP + FP}$$

Recall:

$$r = \frac{TP}{TP + FN}$$

Accuracy:

$$acc = \frac{TP + TN}{TP + TN + FP + FN}$$

F-measure: Harmonic mean of precision and recall

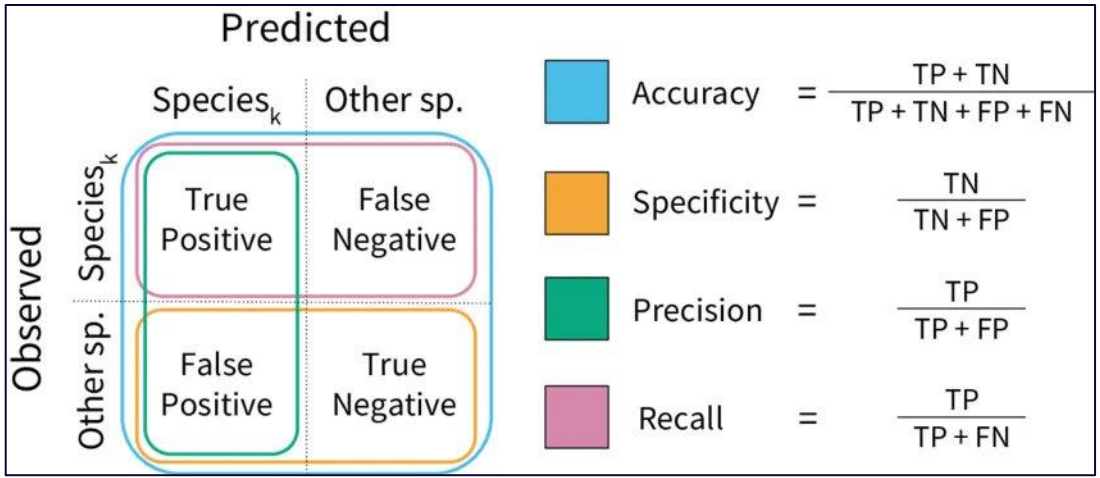
$$F = \frac{1}{\frac{1}{2}\left(\frac{1}{p} + \frac{1}{r}\right)} = \frac{2pr}{p + r}$$

52

- The confusion matrix components (TP, TN, FP, FN) form the basis for computing common classification metrics such as accuracy, precision, recall, and F1-score.

Classification Metrics (Categorical Outputs)(Cont.)

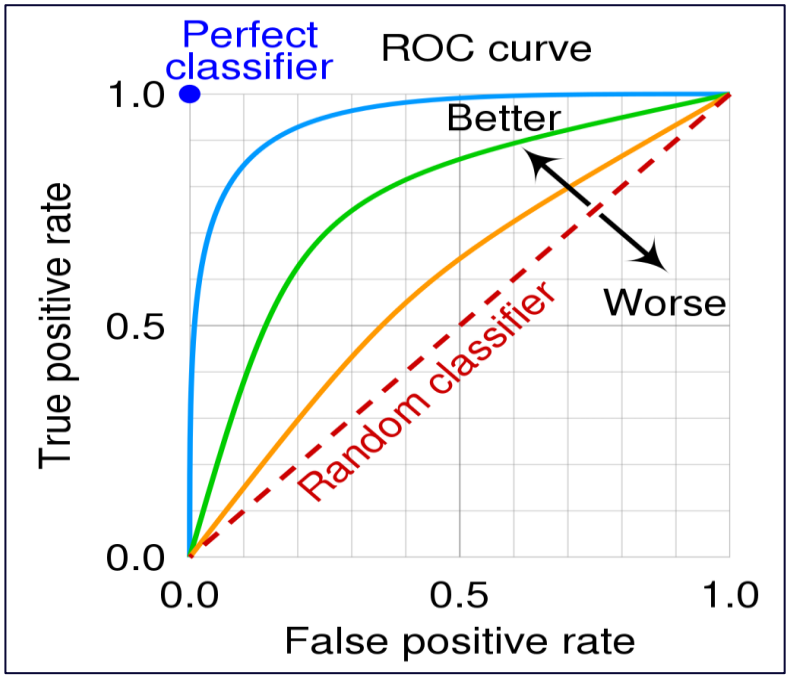
- **Confusion Matrix:**
  - A table that summarizes prediction results and shows:
  1. True Positives (TP)
  2. True Negatives (TN)
  3. False Positives (FP)
  4. False Negatives (FN)
  - It helps analyze the **types of errors made by the model**.



Classification Metrics (Categorical Outputs)(Cont.)

- **ROC Curve (Receiver Operating Characteristic):**
  - Plots the **True Positive Rate (Recall)** against the **False Positive Rate** at different thresholds
- **AUC (Area Under the Curve):**
  - Measures the model's ability to distinguish between classes.
  - Values closer to **1 indicate better classification performance**.
- **When to use it:**
  - Useful when evaluating **binary classifiers**.
  - Helps compare models across different decision thresholds

ROC Curve and AUC



- The ROC curve illustrates the trade-off between the True Positive Rate and the False Positive Rate across different decision thresholds. The Area Under the Curve (AUC) summarizes the model's overall ability to distinguish between classes.

Regression Metrics (Continuous Outputs)

Mean Absolute Error (MAE)

- Average absolute difference between predicted and actual values.
- Easy to interpret and less sensitive to outliers

Mean Squared Error (MSE)

- Average of squared differences between predicted and true values.
- Penalizes larger errors more strongly

Root Mean Squared Error (RMSE)

- Square root of MSE.
- Expressed in the same unit as the target variable, making it easier to interpret

R-squared (R^2)

- Measures how well the model explains the variability of the target variable.
- Values closer to **1 indicate better model fit**.

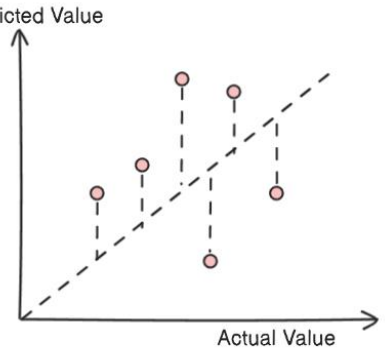
Regression Metrics (Continuous Outputs)(Cont.)

Regression Metrics

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$



$$R^2 = 1 - \frac{SS_{Regression}}{SS_{Total}} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

$$Adjusted\ R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

- Regression metrics quantify the difference between predicted and actual values to measure model prediction error and goodness of fit.

Selecting the Appropriate Metric

- Choosing the correct evaluation metric depends on the nature of the problem:

1. **Imbalanced datasets:** Use Precision, Recall, or F1-score instead of Accuracy.
2. **Binary classification with threshold analysis:** Use ROC-AUC.
3. **Regression with large error sensitivity:** Use RMSE.
4. **Interpretability:** Use MAE because it represents average prediction error

- Selecting the appropriate evaluation metric is essential for accurately measuring model performance and making reliable model comparisons.

.

03 Evaluation Module

Evaluation Module

1. Defining the Model Evaluation Function:

- This cell defines the function **evaluate-model()**, which evaluates a trained classifier using standard metrics such as **accuracy, precision, recall, and F1-score**.
- The function supports both **binary and multi-class classification** and returns the evaluation results in a structured format.

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

def evaluate_model(model, X, y, average_multi="weighted"):
    """
    Evaluate a trained classifier and return standard metrics.
    Supports both binary and multi-class classification.
    """
    y_pred = model.predict(X)
    n_classes = len(np.unique(y))

    acc = accuracy_score(y, y_pred)

    if n_classes == 2:
        precision = precision_score(y, y_pred)
        recall = recall_score(y, y_pred)
        f1 = f1_score(y, y_pred)
    else:
        precision = precision_score(y, y_pred, average=average_multi)
        recall = recall_score(y, y_pred, average=average_multi)
        f1 = f1_score(y, y_pred, average=average_multi)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1_score": f1
    }
```

Evaluation Module (Cont.)

2. Creating a Function to Compare Multiple Models:

- In this cell, we implement the function **compare-models()**, which evaluates several models using the previously defined evaluation function.
- The results are collected and organized into a **comparison table** to facilitate performance analysis

```python
import pandas as pd

def compare_models(models_dict, X, y, average_multi="weighted"):
    """
    Evaluate multiple trained models and return a comparison table.

    models_dict: dictionary in the form
        {"Model Name": trained_model}
    """
    results = []

    for model_name, model in models_dict.items():
        metrics = evaluate_model(model, X, y, average_multi=average_multi)
        results.append([
            model_name,
            metrics["accuracy"],
            metrics["precision"],
            metrics["recall"],
            metrics["f1_score"]
        ])

    return pd.DataFrame(
        results,
        columns=["Model", "Accuracy", "Precision", "Recall", "F1-score"]
    )
```

Evaluation Module (Cont.)

3. Applying the Evaluation Module to the Trained Models:

- Here, we apply the evaluation module to the models trained in the previous section.
- The function evaluates each model and generates a **comparison table** that summarizes their performance using the selected evaluation metrics.

```
models_text = {
    "Logistic Regression": lr_t,
    "Linear SVM": svm_t,
    "Naive Bayes": nb_t,
    "Voting (Hard)": voting_t,
    "Stacking": stack_t
}

text_comparison = compare_models(models_text, X_val_t, y_val_t)
print(text_comparison)
```

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.8732	0.8715	0.8732	0.8720
1	Linear SVM	0.8894	0.8887	0.8894	0.8889
2	Naive Bayes	0.8421	0.8403	0.8421	0.8410
3	Voting (Hard)	0.8950	0.8941	0.8950	0.8944
4	Stacking	0.9036	0.9029	0.9036	0.9031

04 Basic Unit Testing

Basic Unit Testing

- Basic unit testing is used to verify that individual components of the machine learning pipeline work correctly before integration into the full system

- **It helps ensure:**
  - Prediction outputs have the correct shape
  - Evaluation metrics fall within valid ranges
  - Evaluation functions return valid structured outputs
  - Errors can be detected early during development

  - We will implemet basic automated unit tests are applied to validate the evaluation module and improve pipeline reliability.

Types of Unit Testing

Manual Unit Testing

- Tests are performed manually without automation tools
- Time-consuming and difficult to repeat
- Less practical for large or frequently updated projects

Automated Unit Testing

- Tests are executed automatically using code
- Faster, repeatable, and more reliable
- Helps detect bugs early during development

Basic Unit Testing Implementation

- After building the evaluation module, basic automated unit tests are introduced to verify that its outputs are correct, valid, and consistent across different models.

1. Defining Basic Unit Test Functions

- In this cell, we define a set of basic unit test functions for the evaluation module.
- These tests check whether prediction outputs have the correct size, whether evaluation metric values are within the valid range, and whether the evaluation function returns the expected structured output.

```python
def test_prediction_shape(model, X):
    y_pred = model.predict(X)
    assert len(y_pred) == len(X), "Prediction length does not match input size"

def test_metric_range(metrics_dict):
    for metric_name, value in metrics_dict.items():
        assert 0 <= value <= 1, f"{metric_name} is out of the valid range [0, 1]"

def test_evaluation_output_structure(metrics_dict):
    required_keys = {"accuracy", "precision", "recall", "f1_score"}
    assert required_keys.issubset(metrics_dict.keys()), "Missing required evaluation metrics"
```

Basic Unit Testing Implementation (Cont.)

2. Running Unit Tests on the Evaluation Module

- This cell applies the previously defined unit tests to the evaluation module using one trained model.
- It verifies the correctness of prediction size, the validity of evaluation metric values, and the structure of the returned evaluation results

```
# Example: evaluate one trained model
metrics = evaluate_model(lr_t, X_val_t, y_val_t)

# Run tests
test_prediction_shape(lr_t, X_val_t)
test_metric_range(metrics)
test_evaluation_output_structure(metrics)

print("All basic unit tests passed successfully.")
```

Basic Unit Testing Implementation (Cont.)

3. Testing Multiple Models in the Evaluation Pipeline

- In this cell, the unit tests are applied to all trained models in the pipeline, including both single models and ensemble models.
- This ensures that the evaluation module works consistently across all models under the same validation conditions

```
models_to_test = {
    "Logistic Regression": lr_t,
    "Linear SVM": svm_t,
    "Naive Bayes": nb_t,
    "Voting (Hard)": voting_t,
    "Stacking": stack_t
}

for model_name, model in models_to_test.items():
    metrics = evaluate_model(model, X_val_t, y_val_t)
    test_prediction_shape(model, X_val_t)
    test_metric_range(metrics)
    test_evaluation_output_structure(metrics)
    print(f"{model_name}: All tests passed.")
```

- These unit tests improve the reliability of the evaluation pipeline and help detect implementation errors early in the development process.

05 Track Experiments & Configurations

Experiment Tracking

- Experiment tracking is the process of recording machine learning experiments, including model configurations and evaluation results.
- It allows researchers to systematically analyze and compare different models

- **Key elements tracked:**
  - Model configurations (e.g., dataset, number of features)
  - Evaluation metrics (accuracy, precision, recall, F1-score)
  - Experiment results across different models

- **Benefits:**
  - improves reproducibility
  - enables fair comparison between models
  - supports systematic experimentation

Track Experiments & Configurations Implementation

1. Initializing the Experiment Tracking Log

- In this cell, we initialize an **experiment log** using a Pandas DataFrame.
- This table will store the results of different machine learning experiments, including model names, dataset information, number of features, and evaluation metrics.

```
import pandas as pd

experiment_log = pd.DataFrame(columns=[
    "Model",
    "Dataset",
    "Num_Features",
    "Accuracy",
    "Precision",
    "Recall",
    "F1-score"
])

experiment_log
```

Track Experiments & Configurations Implementation (Cont.)

2. Tracking Experiments and Model Configurations

- In this cell, we record both experiment results and model configurations.
- For each trained model, we compute the evaluation metrics and store them along with configuration information such as dataset name and number of selected features.
- This structured logging enables systematic comparison between models and improves experiment reproducibility

.

Track Experiments & Configurations Implementation (Cont.)

2. Tracking Experiments and Model Configurations (Cont.)

```
dataset_name = "Text"
num_features = X_text_sel.shape[1]

models_text = {
    "Logistic Regression": lr_t,
    "Linear SVM": svm_t,
    "Naive Bayes": nb_t,
    "Voting (Hard)": voting_t,
    "Stacking": stack_t
}

for model_name, model in models_text.items():

    metrics = evaluate_model(model, X_val_t, y_val_t)

    experiment_log.loc[len(experiment_log)] = [
        model_name,
        dataset_name,
        num_features,
        metrics["accuracy"],
        metrics["precision"],
        metrics["recall"],
        metrics["f1_score"]
    ]

experiment_log
```

F1-score	Recall	Precision	Accuracy	Num_Features	Dataset	Model
0.86	0.87	0.86	0.87	250	Text	Logistic Regression
0.88	0.89	0.88	0.89	250	Text	Linear SVM
0.83	0.84	0.83	0.84	250	Text	Naive Bayes
0.89	0.90	0.89	0.90	250	Text	Voting (Hard)
0.90	0.91	0.90	0.91	250	Text	Stacking

Track Experiments & Configurations Implementation (Cont.)

3. Saving Experiment Results for Reproducibility

- In this cell, the experiment log is saved as a **CSV file**.
  Saving experiment results allows us to reproduce experiments later, analyze model performance over time, and maintain a persistent record of the evaluation pipeline

```
experiment_log.to_csv("experiment_results.csv", index=False)

print("Experiment results saved successfully.")
```

06 Reproducible Evaluation Pipeline

Reproducible Evaluation Pipeline

- By combining evaluation modules, unit testing, and experiment tracking, we build a reproducible evaluation pipeline.

- This pipeline ensures that:.
  - models are evaluated consistently
  - experiment configurations are recorded
  - results can be reproduced and compared easily.

  - Such practices improve the reliability and transparency of machine learning experimentation.

.